



At this point in the course, you've learned quite a bit about TypeScript — its basics as well as some more advanced features. Up until now, we've always worked in relatively simple demo projects and project setups.

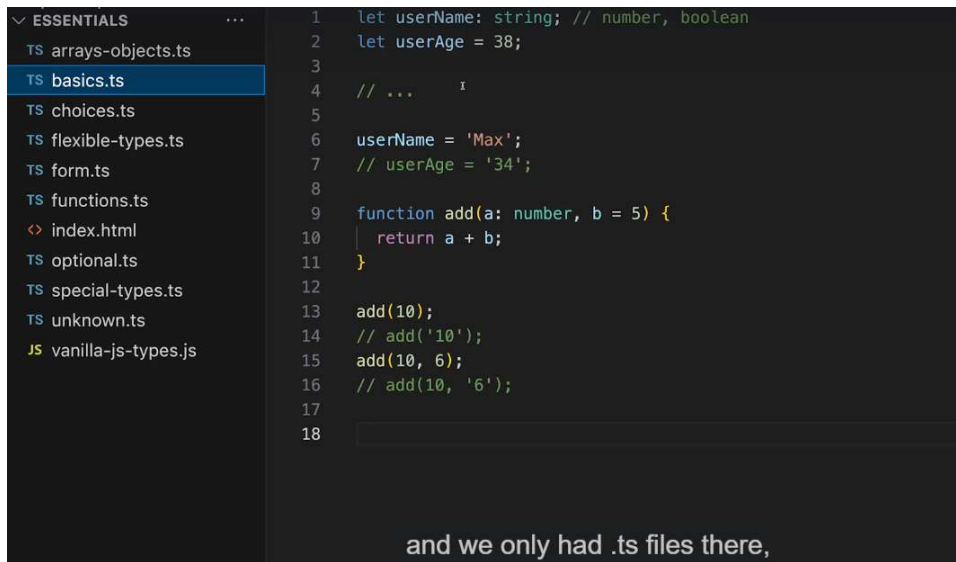
Of course, in real-world development you'll often work on more complex projects that consist of multiple files and aren't limited to just TypeScript. You'll typically also have HTML, CSS, and many other types of files in your project.

That's why in this section we'll explore how to build such TypeScript projects and, more generally, how to work with larger and more complex TypeScript codebases. For this, we'll go beyond the `tsc` command (the TypeScript Compiler we installed earlier in the course) and look at alternative build tools. Most importantly, we'll take a closer look at **Vite**.

The Problem With The Typescript Compiler

For this section, I've created a brand new, very basic demo project (which you'll find attached). It's still not a super complex project — it only consists of three source files — but it's an important step forward. Unlike most of the projects we've seen so far in the course, this one does **not** consist only of `.ts` files. It also includes HTML and CSS files.

This is a key difference because up until now, we've mostly been building **Node.js** projects — applications meant to run with Node.js, containing only TypeScript files that get compiled to JavaScript.



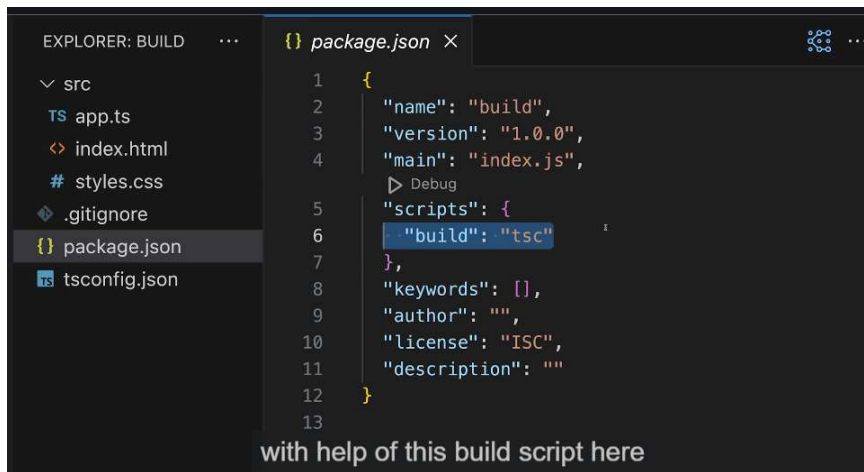
The screenshot shows a VS Code editor with a sidebar on the left containing a file explorer. The 'ESSENTIALS' folder is expanded, showing several TypeScript files: arrays-objects.ts, basics.ts (selected), choices.ts, flexible-types.ts, form.ts, functions.ts, index.html, optional.ts, special-types.ts, unknown.ts, and vanilla-js-types.js. The main editor displays the contents of basics.ts, which includes variable declarations for userName and userAge, a function add, and several function calls. The code is as follows:

```
1 let userName: string; // number, boolean
2 let userAge = 38;
3
4 // ...
5
6 userName = 'Max';
7 // userAge = '34';
8
9 function add(a: number, b = 5) {
10   return a + b;
11 }
12
13 add(10);
14 // add('10');
15 add(10, 6);
16 // add(10, '6');
17
18
```

Below the code, the text "and we only had .ts files there," is written.

Now, however, we're building a **frontend / client-side project** that is intended to run in the browser. That's why we have HTML and CSS files in addition to TypeScript. (Of course, even Node.js projects can include more than just TypeScript files, but this demo focuses on the browser environment.)

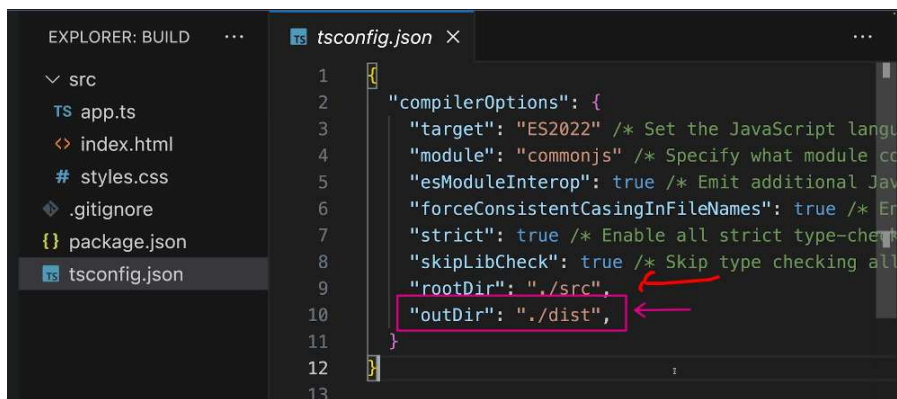
In this example, I'm running into a problem: I'm trying to build this project using the build script we had before — but it no longer works properly because the project now contains more than just TypeScript files



The screenshot shows a VS Code editor with a sidebar on the left containing a file explorer. The 'EXPLORER: BUILD' folder is expanded, showing several files: app.ts, index.html, styles.css, .gitignore, package.json (selected), and tsconfig.json. The main editor displays the contents of package.json, which includes a 'build' script set to 'tsc'. The code is as follows:

```
1 {
2   "name": "build",
3   "version": "1.0.0",
4   "main": "index.js",
5   "scripts": {
6     "build": "tsc"
7   },
8   "keywords": [],
9   "author": "",
10  "license": "ISC",
11  "description": ""
12 }
13
```

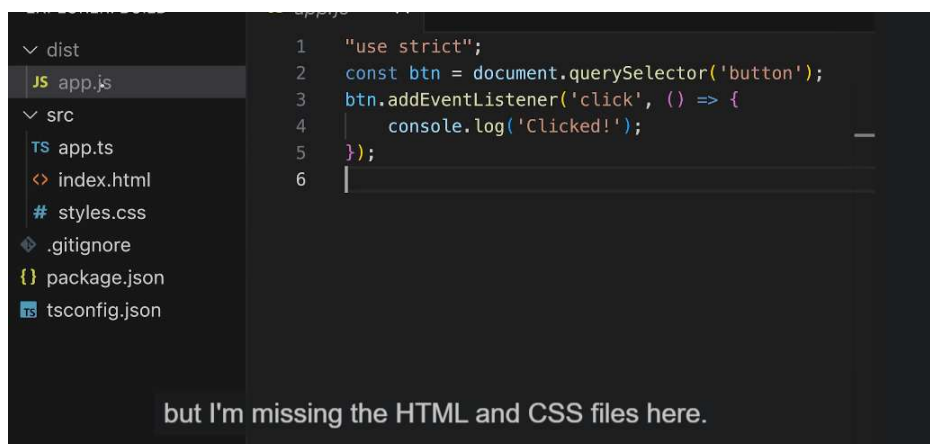
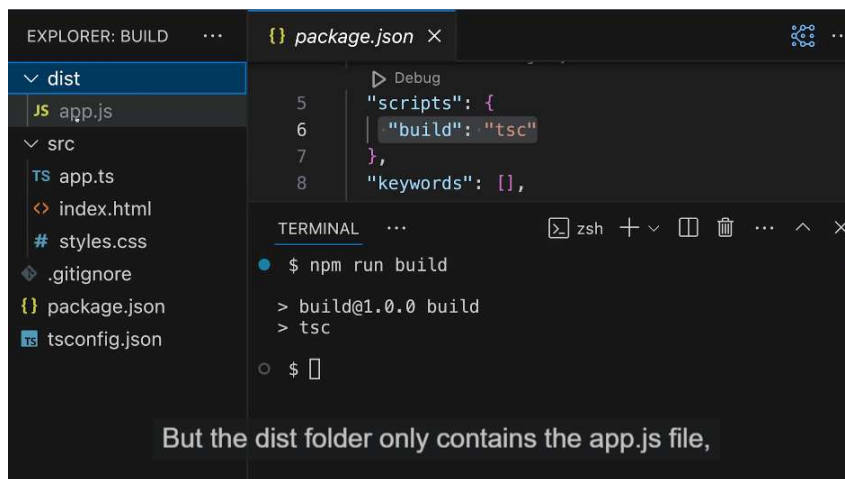
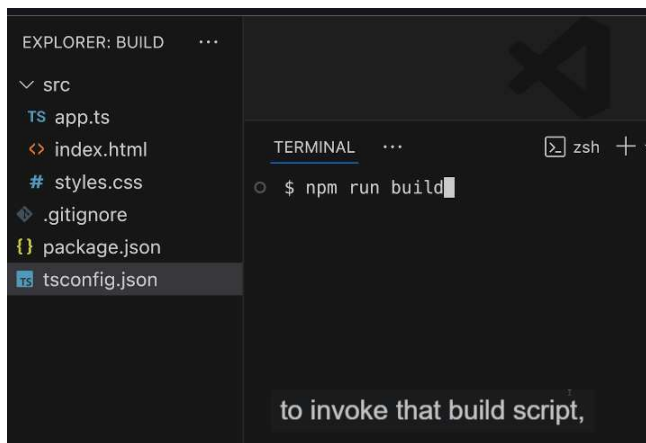
Below the code, the text "with help of this build script here" is written.



The screenshot shows a VS Code editor with a sidebar on the left containing a file explorer. The 'EXPLORER: BUILD' folder is expanded, showing several files: app.ts, index.html, styles.css, .gitignore, package.json, and tsconfig.json (selected). The main editor displays the contents of tsconfig.json, which includes compiler options. The code is as follows:

```
1 {
2   "compilerOptions": {
3     "target": "ES2022" /* Set the JavaScript language target */
4     "module": "commonjs" /* Specify what module code generator to use */
5     "esModuleInterop": true /* Emit additional JavaScript to ease support for importing CommonJS modules. This enables 'allowSyntheticDefaultImports' by default, which might not be desirable in all environments. */
6     "forceConsistentCasingInFileNames": true /* Ensure the casing in filenames is consistent with the module system. */
7     "strict": true /* Enable all strict type-checking options. */
8     "skipLibCheck": true /* Skip type checking of library files. */
9     "rootDir": "./src",
10    "outDir": "./dist",
11  }
12 }
13
```

Two red arrows point to the 'rootDir' and 'outDir' properties in the 'compilerOptions' object.



But I'm trying to build this project with the help of this build script — using the same `tsc` command we've used before in the terminal. I'm executing it as part of an npm script. That's why I also have a `tsconfig.json` file here, which is configured to point to the `src` directory as the source and output the compiled files into a `dist` directory.

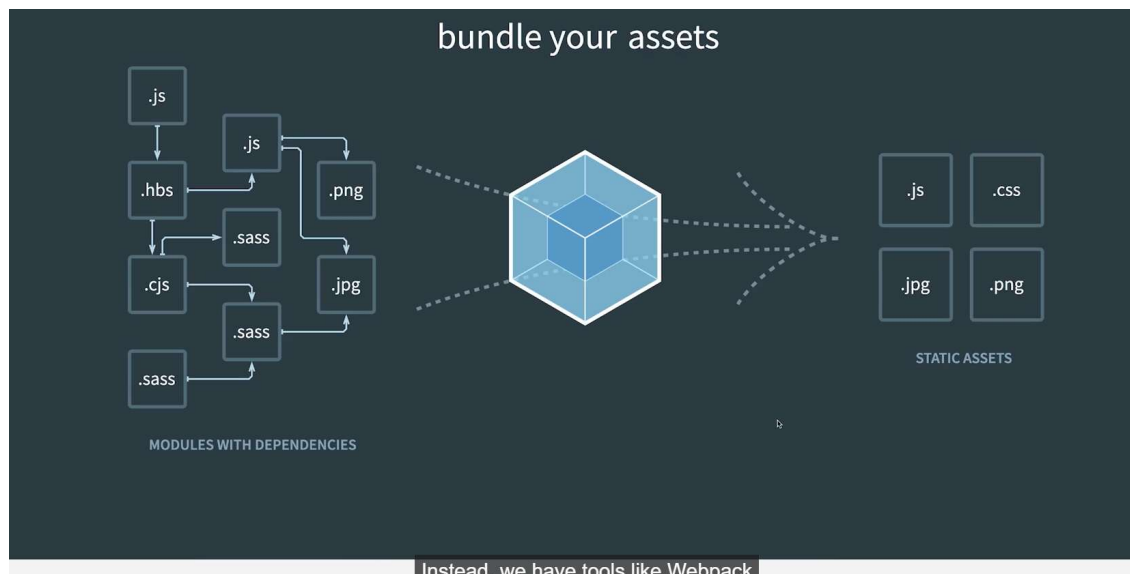
This is a common setup you might use in such projects. So we can run `npm run build` to trigger the build script I defined in `package.json`. When I do that, everything appears to work — we get a `dist` folder. However, inside that `dist` folder, we only find the compiled `app.js` file. Even though I didn't use much TypeScript in this example (just a small detail like the non-null assertion operator), the compilation succeeded — but the HTML and CSS files are completely missing.

If I were to deploy the content of this `dist` folder right now, I wouldn't have a working website, because I'm only left with the JavaScript file.

This highlights one of the fundamental problems when trying to build frontend projects using only the TypeScript compiler (`tsc`).

At its core, **tsc is just a TypeScript compiler**. It's designed to compile TypeScript code into JavaScript — it does **not** handle building entire web projects. It won't copy your HTML or CSS files, nor will it optimize or process other assets. That's the problem we're facing here, and it's exactly why we need a better solution.

Building with Webpack or ESBUILD



Instead, we have tools like Webpack

that aim to help us with building and managing projects,

web projects that consist of all kinds of different files.

And Webpack is a very popular solution,

because it is a tool that can handle all kinds of file types

and bundle them together and optimize them, transform them,

and then spit out a folder

that contains all those changed and transformed

and build files.

And Webpack is still very popular today,

and it especially has been super popular in the past.

Latest Sponsors

Now, working with Webpack,

that would be a topic on its own.

webpack 5

Print Section

Concepts

- Entry
- Output
- Loaders
- Plugins
- Mode
- Browser Compatibility
- Environment

- Entry Points
- Output
- Loaders
- Plugins
- Configuration
- Modules
- Module Resolution
- Module Federation
- Dependency Graph
- Targets
- The Manifest
- Hot Module Replacement
- Why webpack
- Under The Hood

Concepts

At its core, **webpack** is a *static module bundler* for modern JavaScript applications. When webpack processes your application, it internally builds a *dependency graph* from one or more *entry points* and then combines every module your project needs into one or more *bundles*, which are static assets to serve your content from.

Tip

Learn more about JavaScript modules and webpack modules [here](#).

Since version 4.0.0, **webpack does not require a configuration file** to bundle your project. Nevertheless, it is *incredibly configurable* to better fit your needs.

To get started you only need to understand its **Core Concepts**:

- [Entry](#)
- [Output](#)
- [Loaders](#)
- [Plugins](#)
- [Mode](#)
- [Browser Compatibility](#)

This document is intended to give a **high-level** overview of these concepts, while providing links to detailed concept-specific use cases.

You could create an entire course about Webpack, for a better understanding of the ideas behind module bundlers and how they work under the hood, consult these resources:

- [Manually Bundling an Application](#)

about using it, configuring it, and so on.

- [Manually Bundling an Application](#)

But thankfully, there definitely already exists

courses for that.

Manually Bundling an Application

- [Manually Bundling an Application](#)

you typically don't need to do that,

- [Manually Bundling an Application](#)

Live Coding a Basic Module Bundler

- [Manually Bundling an Application](#)

because you'll often work with project setups

- [Manually Bundling an Application](#)

where such a Webpack configuration

- [Manually Bundling an Application](#)

has already been created for you.

- [Manually Bundling an Application](#)

You might have a dedicated team member

- [Manually Bundling an Application](#)

if you're working in a bigger company

and on more complex projects that handles that.

And as a TypeScript Developer, you definitely don't need to

- Browser Compatibility

to know how to configure Webpack, as, detailed concept-specific use cases.

In addition, Webpack, as I mentioned,

- Browser Compatibility

still is super popular and gets used a lot, as, detailed concept-specific use cases.

- Browser Compatibility

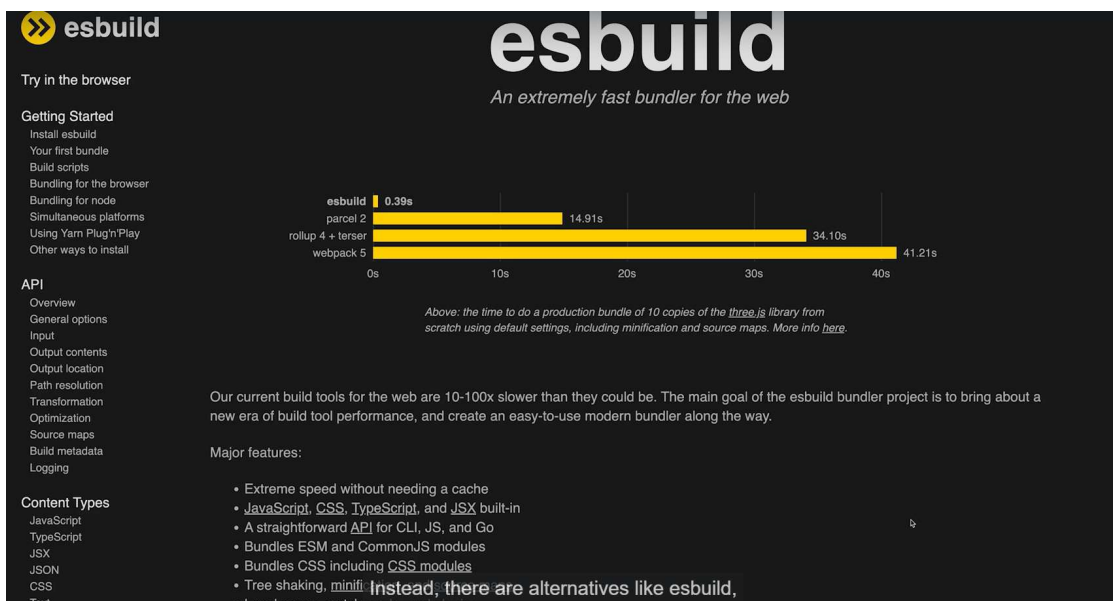
but it's not the most modern as, detailed concept-specific use cases.

- Browser Compatibility

or trending tool to be used as, detailed concept-specific use cases.

- Mode

for managing projects and for compiling files. This document is intended to give a high-level overview of these concepts, while allowing



- Bundles ESM and CommonJS modules

which have become extremely popular over the past years,

- Local server, watch mode, and plugins

mainly because of their performance.

files CSS including CSS modules

shaking, minification. And esbuild would be a popular alternative to Webpack, local server, watch mode, and plugins.

which you can also use to build and optimize and compile

and change and copy all kinds of files.

So you could use esbuild or Webpack

and configure them in your project

to manage how all the files

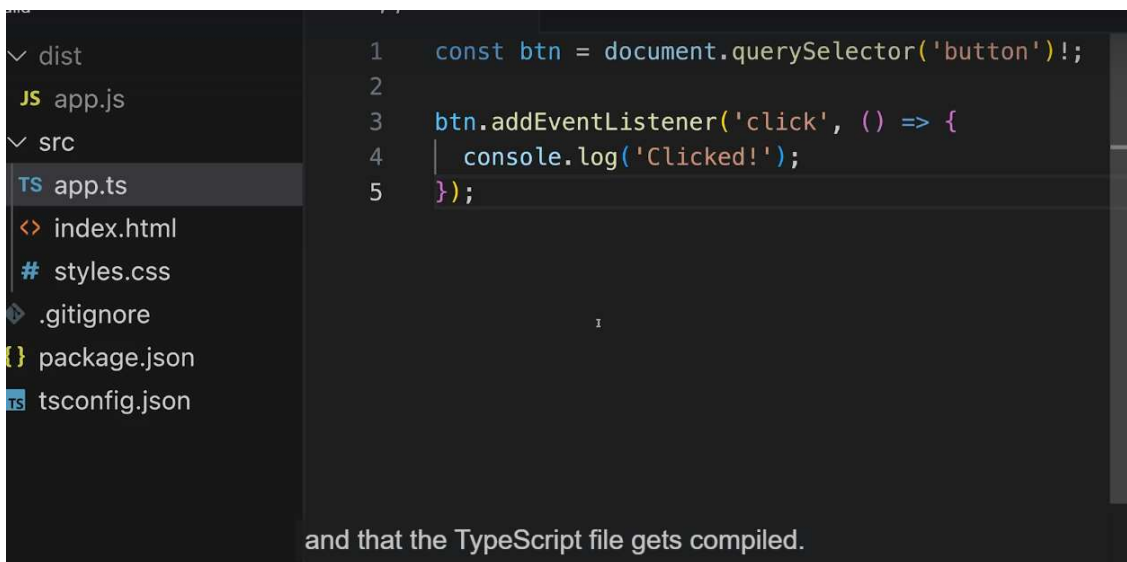
in the source folder are handled.

And to manage that, the HTML file gets copied

to the dist folder, the CSS file gets copied too,



and maybe also gets transformed to support older browsers



Try in the browser

Getting Started

- Install esbuild
- Your first bundle
- Build scripts
- Bundling for the browser
- Bundling for node
- Simultaneous platforms
- Using Yarn Plug'n'Play
- Other ways to install

API

- Overview
- General options
- Input
- Output contents
- Output location
- Path resolution
- Transformation
- Optimization
- Source maps
- Build metadata
- Logging

Content Types

- JavaScript
- TypeScript
- JSX
- JSON
- CSS
- Text

Tool	Time
esbuild	0.39s
parcel 2	14.91s
rollup 4 + terser	34.10s
webpack 6	41.21s

Above: the time to do a production bundle of 10 copies of the `three.js` library from scratch using default settings, including minification and source maps. More info [here](#).

Our current build tools for the web are 10-100x slower than they could be. The main goal of the esbuild bundler project is to bring about a new era of build tool performance, and create an easy-to-use modern bundler along the way.

Major features:

- Extreme speed without needing a cache
- JavaScript, CSS, TypeScript, and JSX built-in
- A straightforward API for CLI, JS, and Go
- Bundles ESM and CommonJS modules
- Bundles CSS including [CSS modules](#)
- Tree shaking, [minification](#), and [source maps](#)
- [Local server](#), [watch mode](#), and [plugins](#)

Check out the [getting started](#) instructions if you want to give esbuild a try.

Because both esbuild and Webpack

also support TypeScript compilation.

but again, you typically don't need to become

Syntax feature Example Notes

a Webpack or esbuild expert,

because typically, you'll work with projects

where these tools have already been set up for you.

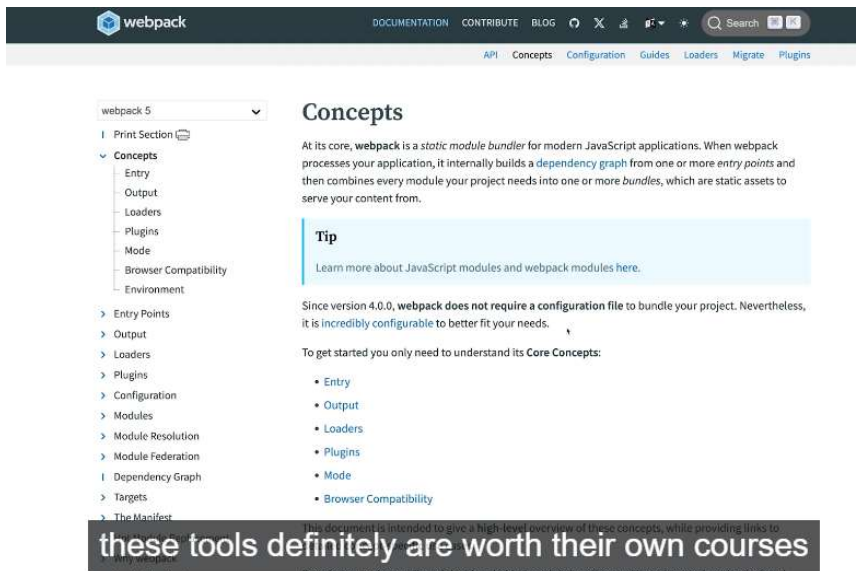
And I'll show you one extremely valuable and popular tool

for creating such projects in the next lecture.

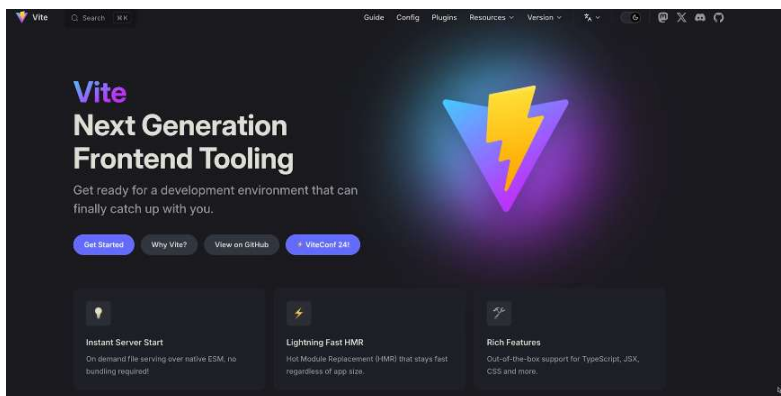
Introducing Vite

So you could configure your own project with ESBUILD or Webpack to handle all these different files. But as I mentioned, you typically don't need to do that. And if you do, well,

these tools definitely are worth their own courses



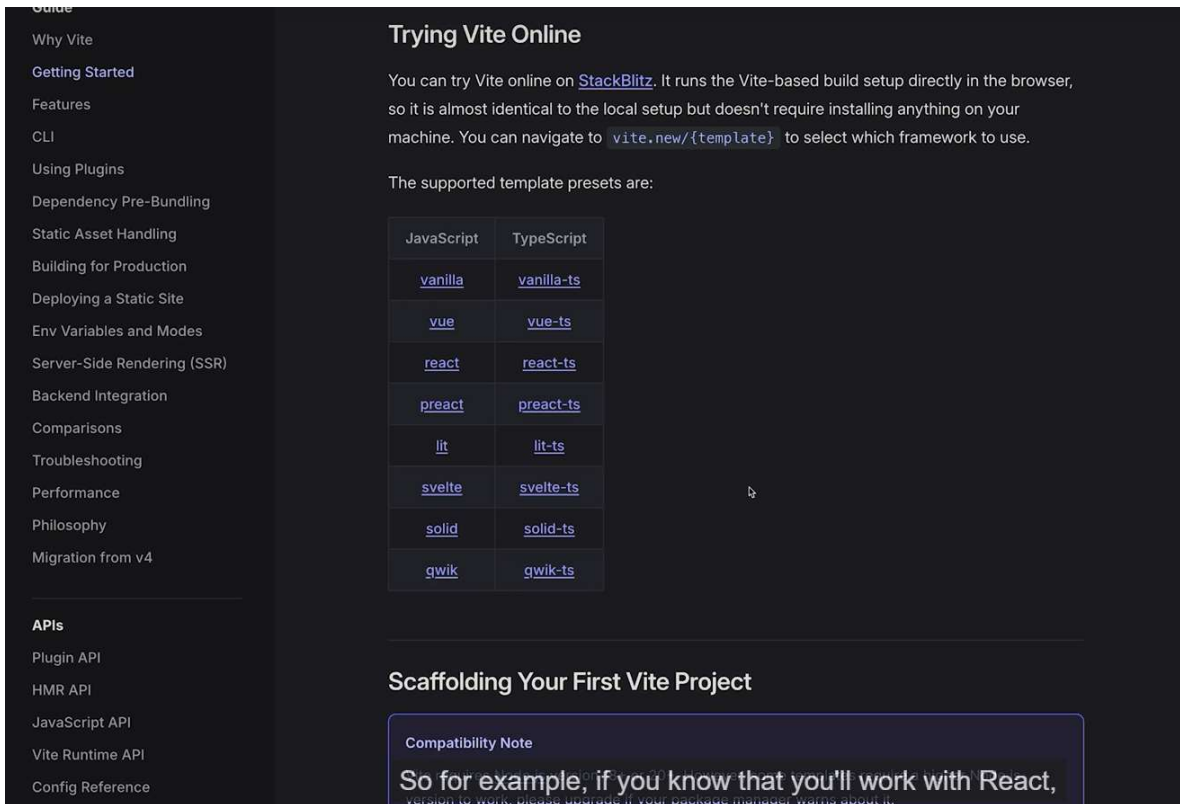
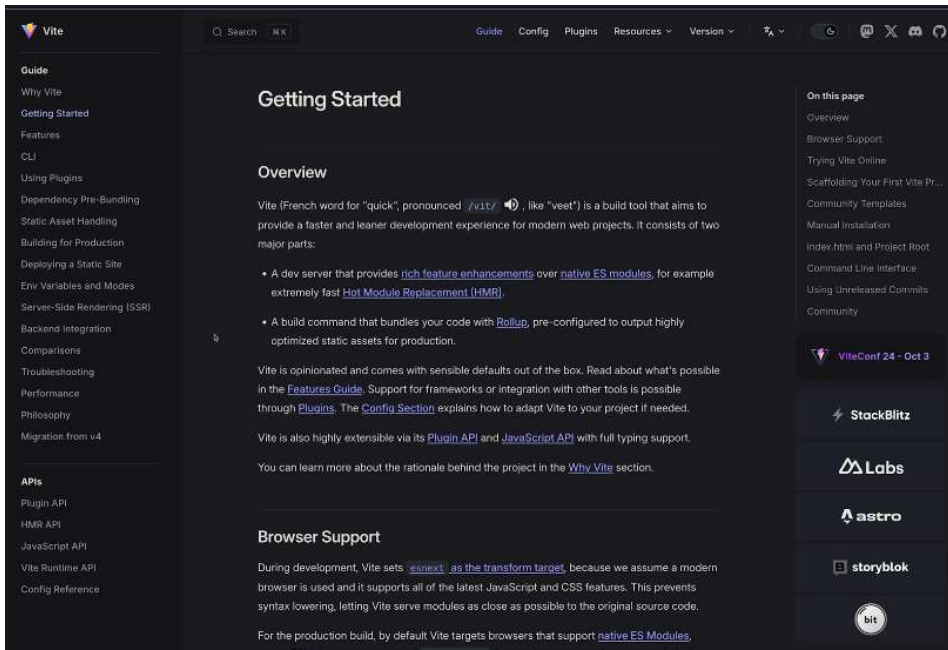
A super popular tool
for building all kinds of web dev projects
that are already set up to support TypeScript,
file optimization, CSS optimization and much, much more.

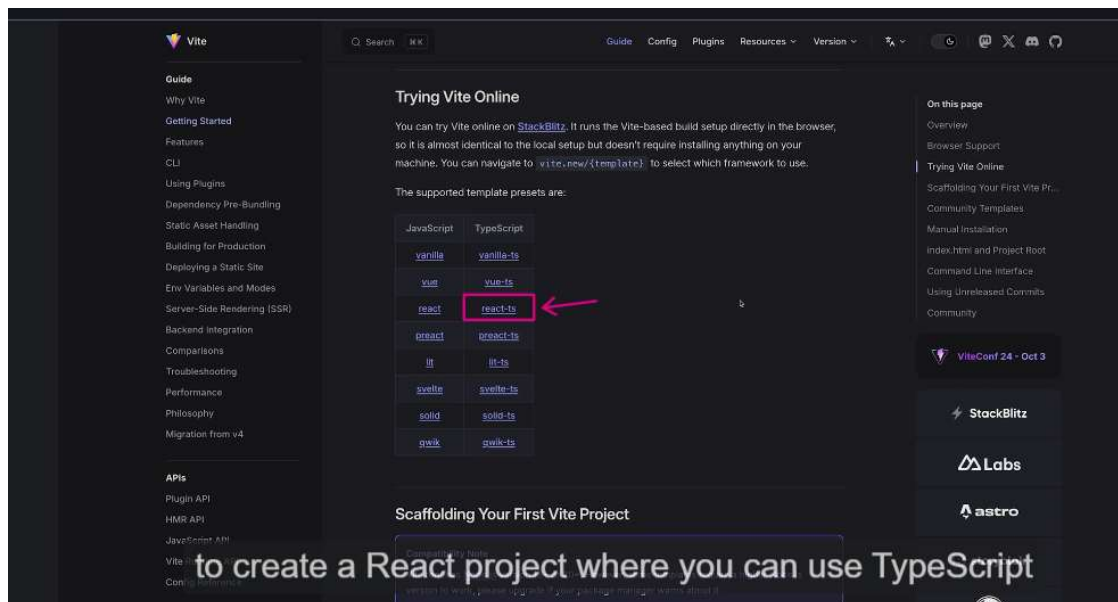


Now, you can learn more about Vite
in their official documentation.

A link can be found attached to this lecture.
But in the end, Vite is a tool that you can use
to create projects that come with ESBUILD

or a similar build tool built-in
and that handle all the copying
and optimization of the various code files
you might be working with in your project.





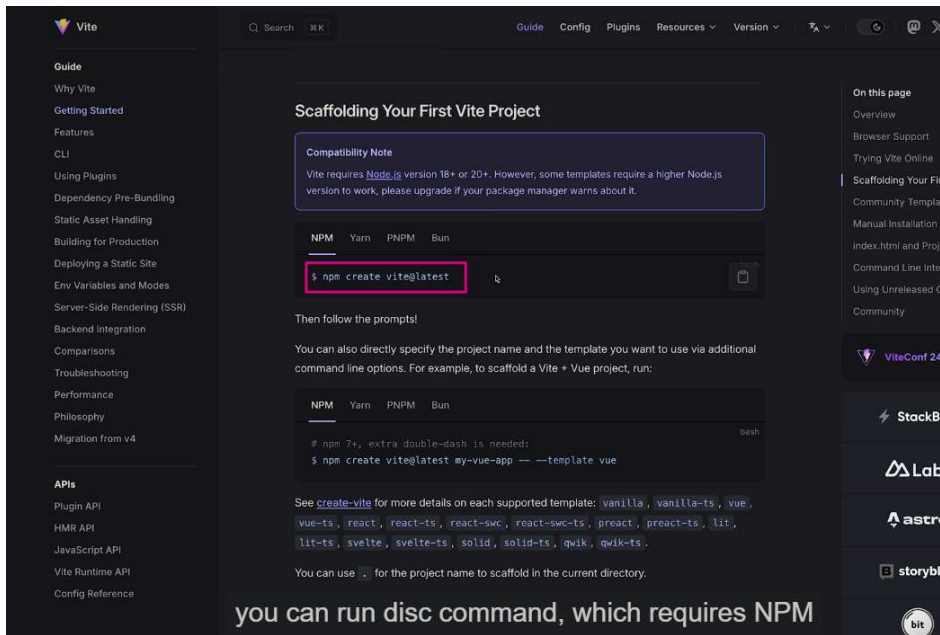
So for example, if you know that you'll work with React, a popular frontend JavaScript library, which also supports TypeScript or which you can also use with TypeScript,

if you know that, well, you can use this preset here supported by Vite to create a React project where you can use TypeScript

and where you can then run one simple command to get a deployable JavaScript-based React project. Now, we'll take a look at React TypeScript a little bit later in this course as one example of using TypeScript.

For now, we'll stick to Vanilla JavaScript and we'll see how we can create a project that supports more than just .TS files, for example, also HTML files with Vite.

Using Vite



to be installed on your machine,

which you should already have installed anyways.

**So you should go to your terminal
or command prompt on your machine,
navigate into the folder where you want to create
your new TypeScript-based web project,
and then run npm create vite@latest.**

```
npm create vite@latest
```

```
$ npm create vite@latest
Need to install the following packages:
create-vite@5.5.2
Ok to proceed? (y)
```

When you run this command,

```
> npx
> create-vite

? Project name: > vite-project
```

and then you'll be asked how you want to name your project.

```
> npx
> create-vite

? Project name: > vite-demo-ts
```

But of course the name is entirely up to you.

```
> npx
> create-vite

✓ Project name: ... vite-demo-ts
? Select a framework: > - Use arrow-keys. Return to submit.
> Vanilla
  Vue
  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
  Thereafter, you have to choose which preset you want to use,
```

So in my case, I want to use a vanilla JavaScript project,

so a frontend web project that uses just JavaScript,

no extra framework or library like React.

But I want to write TypeScript code during development.

And then of course I want to have that code compiled
to JavaScript code for production.

So I'll choose TypeScript here.

```
> npx
> create-vite

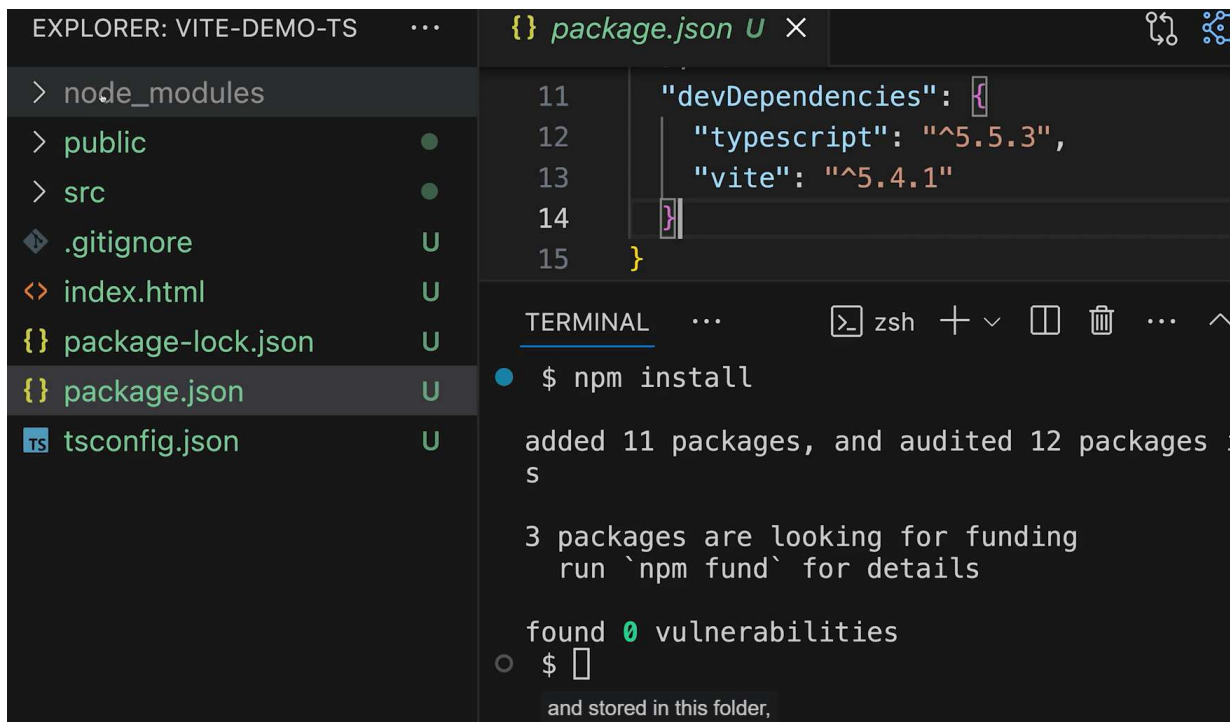
✓ Project name: ... vite-demo-ts
✓ Select a framework: > Vanilla
? Select a variant: > - Use arrow-keys. Return to submit.
>   TypeScript
    JavaScript
```

Done. Now run:

```
cd vite-demo-ts
npm install
npm run dev
```

To build this project, type, 'npm install'

This ensure that all the dependencies that are mentioned in 'package.json' folder are installed

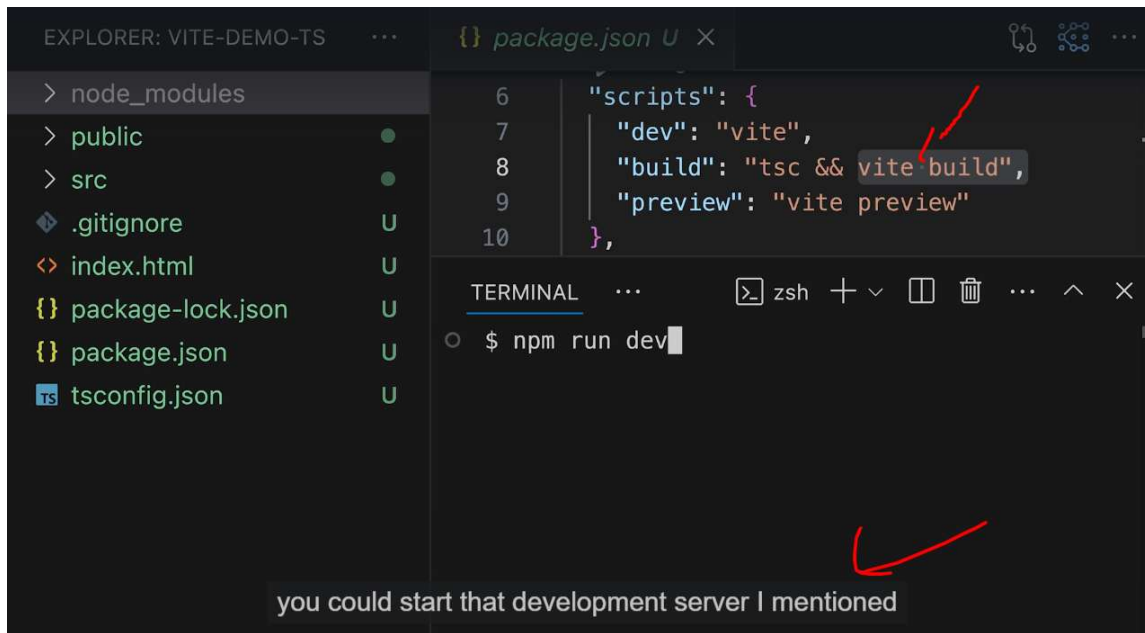


The screenshot shows the VS Code interface with the Explorer on the left, the package.json file open in the editor, and the Terminal at the bottom. The Explorer shows the project structure with folders like node_modules, public, and src, and files like .gitignore, index.html, package-lock.json, package.json, and tsconfig.json. The package.json file shows the devDependencies section with typescript and vite. The Terminal shows the output of the npm install command, indicating that 11 packages were added and 12 packages were audited, with no vulnerabilities found.

```
EXPLORER: VITE-DEMO-TS  ...
> node_modules
> public
> src
.gitignore U
index.html U
package-lock.json U
package.json U
tsconfig.json U

package.json
11 "devDependencies": {
12   "typescript": "^5.5.3",
13   "vite": "^5.4.1"
14 }
15 }

TERMINAL  ...  zsh
$ npm install
added 11 packages, and audited 12 packages
3 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
$
```

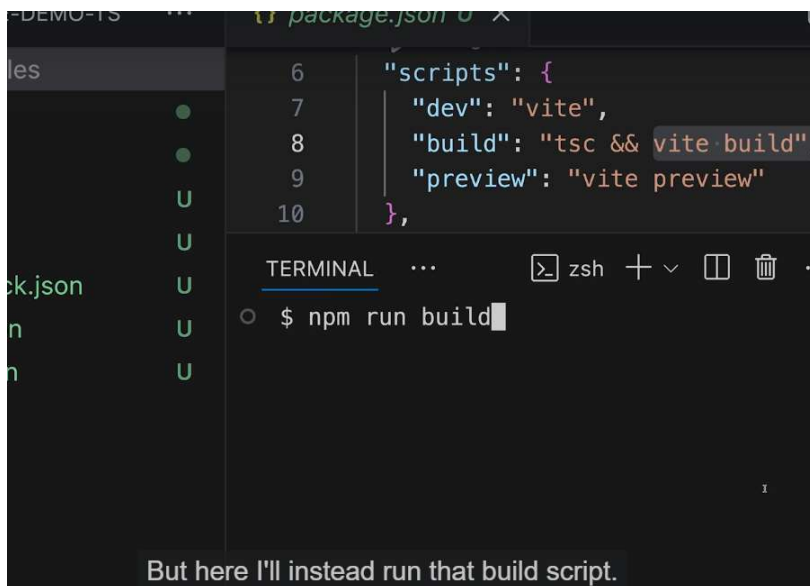


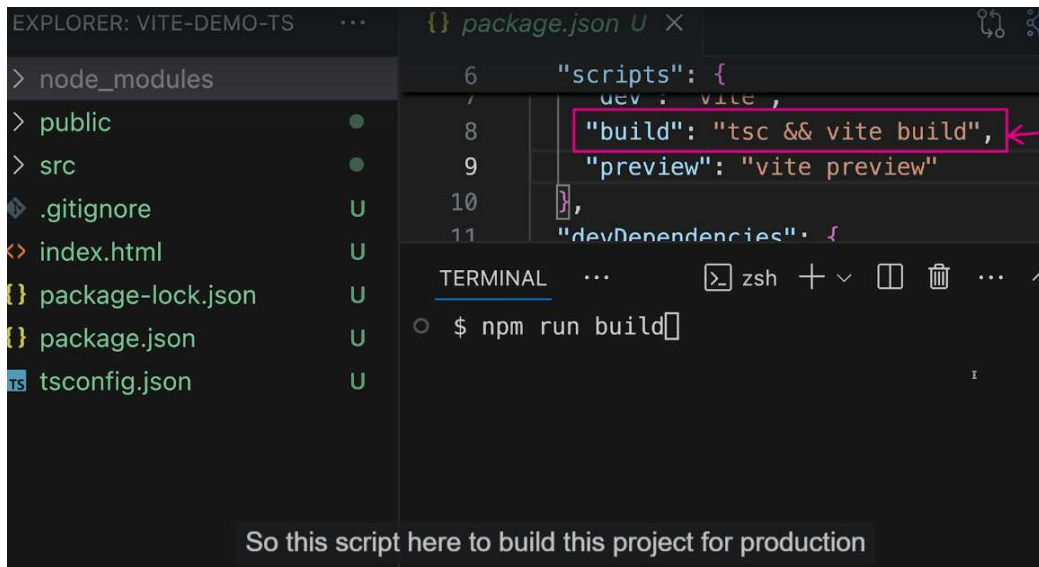
And that would then open up a website

which you could visit in your browser

and start a process which watches your code files

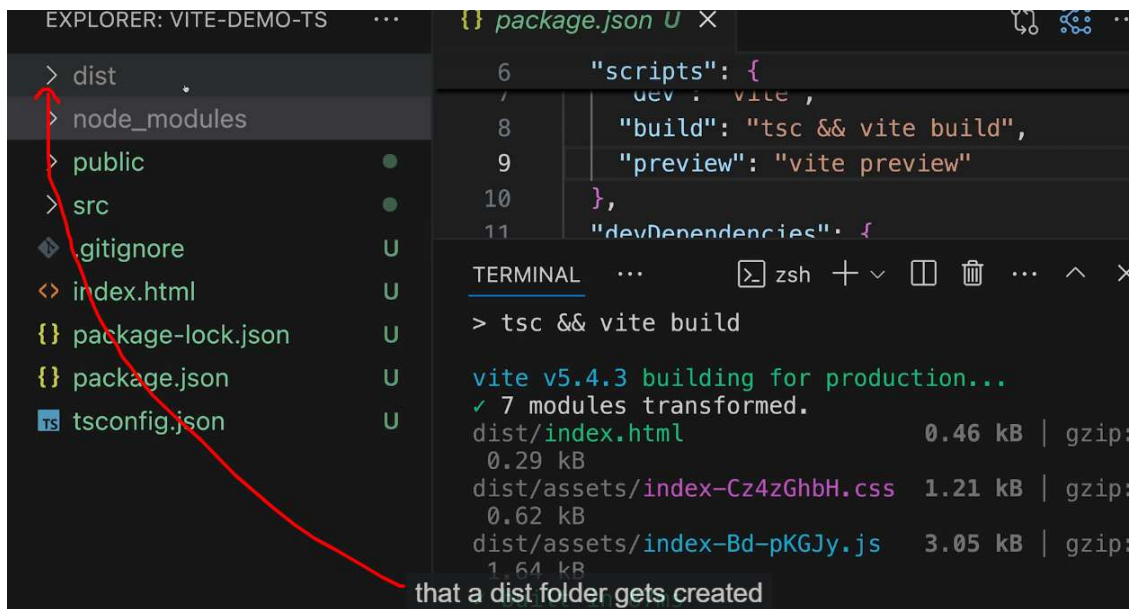
to update that website whenever you make changes.

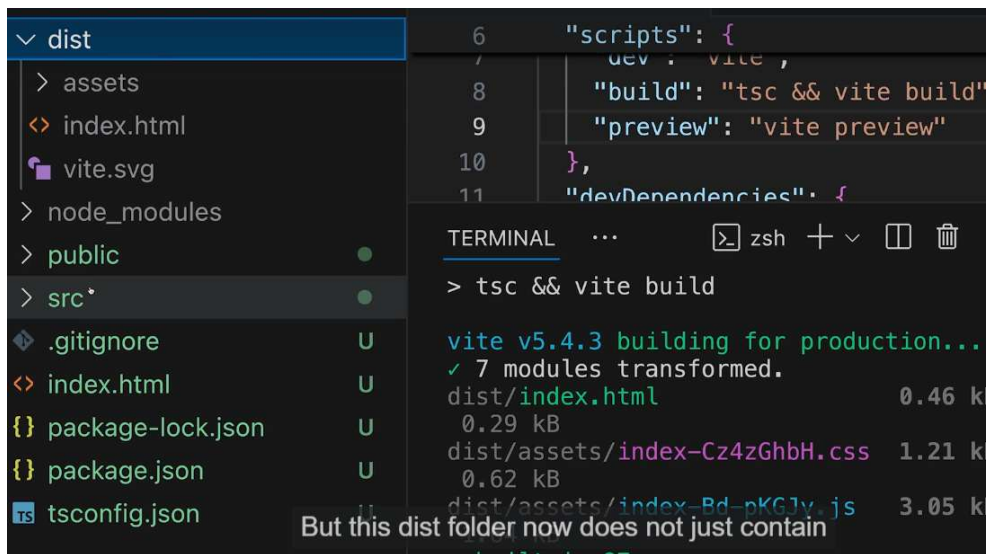




to get a deployable folder.

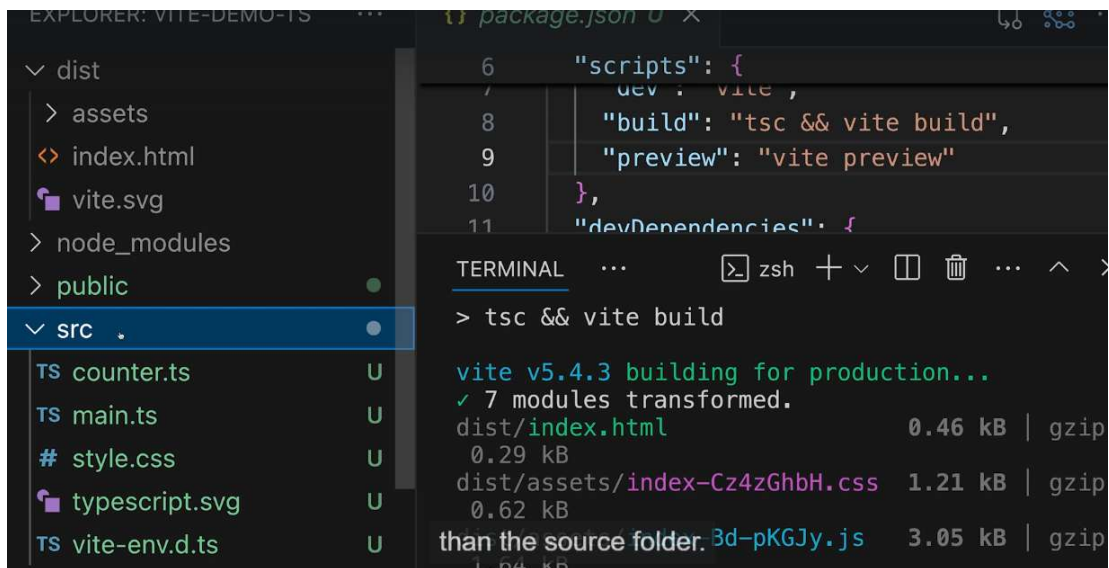
And if I do that, you'll see





the compiled TypeScript files.

Instead it has a totally different structure



It, for example, contains the index HTML file, js

which is not even part of the source folder,

EXPLORER: VITE-DEMO-TS ...

dist

- assets
- index.html
- vite.svg

node_modules

public

src

.gitignore

index.html

package-lock.json

package.json

tsconfig.json

index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg">
```

TERMINAL ... zsh + -

```
> tsc && vite build

vite v5.4.3 building for production...
✓ 7 modules transformed.
dist/index.html                                0.46 kB | gzip
0.29 kB
dist/assets/index-Cz4zGhbH.css                 1.21 kB | gzip
0.62 kB
dist/assets/index-Bd-nKGJy.js                  3.05 kB | gzip
✓ built in 87ms
```

but which is part of the root folder.

dist

- assets
- index.html
- vite.svg

node_modules

public

src

.gitignore

index.html

package-lock.json

package.json

tsconfig.json

index.html

```
vite v5.4.3 building for p
✓ 7 modules transformed.
dist/index.html
0.29 kB
dist/assets/index-Cz4zGhbH
0.62 kB
```

Nonetheless, this file here, in the end

EXPLORER: VITE-DEMO-TS ...

dist

- assets
- index.html
- vite.svg

node_modules

public

src

.gitignore

index.html

package-lock.json

package.json

tsconfig.json

index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg">
```

TERMINAL ... zsh + -

```
> tsc && vite build

vite v5.4.3 building for produ
✓ 7 modules transformed.
dist/index.html
0.29 kB
dist/assets/index-Cz4zGhbH.css
0.62 kB
dist/assets/index-Bd-nKGJy.js
3.05 kB
```

gets copied into the dist folder by Vite

EXPLORER: VITE-DEMO-TS ...

- dist
 - assets
 - index.html
 - vite.svg
- node_modules
- public
- src
- .gitignore
- index.html
- package-lock.json
- package.json
- tsconfig.json

index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <link rel="icon" type="image/svg+xml" href="/vite.svg">
```

TERMINAL ... zsh + v

```
> tsc && vite build

vite v5.4.3 building for production...
✓ 7 modules transformed.
dist/index.html                0.29 kB
dist/assets/index-Cz4zGhbH.css 0.62 kB
dist/assets/index-Bd-pKGJy.js  3.05 kB
```

so that we can take the contents of the dist folder,

put them onto a server.

and have our running website there.

EXPLORER: VITE-DEMO-TS ...

- dist
 - assets
 - index-Bd-pKGJy.js
 - index-Cz4zGhbH.css
 - index.html
 - vite.svg
- node_modules
- public
- src
- .gitignore
- index.html

index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg">
```

TERMINAL ... zsh + v

```
> tsc && vite build

vite v5.4.3 building for production...
✓ 7 modules transformed.
dist/index.html                0.46 kB
dist/assets/index-Cz4zGhbH.css 1.21 kB
dist/assets/index-Bd-pKGJy.js  3.05 kB
```

And it's then in the assets folder

EXPLORER: VITE-DEMO-TS ...

- dist
 - assets
 - index-Bd-pKGJy.js
 - index-Cz4zGhbH.css
 - index.html
 - vite.svg
- node_modules
- public
- src
- .gitignore
- index.html
- package-lock.json

index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg">
```

TERMINAL ... zsh + v

```
> tsc && vite build

vite v5.4.3 building for production...
✓ 7 modules transformed.
dist/index.html                0.46 kB | gzip
dist/assets/index-Cz4zGhbH.css 1.21 kB | gzip
dist/assets/index-Bd-pKGJy.js  3.05 kB | gzip
```

where we got this weirdly named JavaScript file,


```
EXPLORER: VITE-DEMO-TS ... JS index-Bd-pKGJy.js X ...
dist
  assets
    JS index-Bd-pKGJy.js
    # index-Cz4zGhbH.css
  index.html
  vite.svg
node_modules
public
src
.gitignore
index.html
package-lock.json
package.json

1 (function(){const c=document.createElement(
2   <div>
3     <a href="https://vitejs.dev" target="_b
4     |  ,
6     <a href="https://www.typescriptlang.org
7     | 
9     <h1>Vite + TypeScript</h1>
10    <div class="card">
11      <button id="counter" type="button"></
12    </div>
13    <p class="read-the-docs">
14      Click on the Vite and TypeScript logo
15  )})
```

which in the end contains the compiled TypeScript code.

```
EXPLORER: VITE-DEMO-TS ... JS index-Bd-pKGJy.js X ...
dist
  assets
    JS index-Bd-pKGJy.js
    # index-Cz4zGhbH.css
  index.html
  vite.svg
node_modules
public
src
.gitignore
index.html
package-lock.json
package.json

1 (function(){const c=document.createElement(
2   <div>
3     <a href="https://vitejs.dev" target="_b
4     |  ,
6     <a href="https://www.typescriptlang.org
7     | 
9     <h1>Vite + TypeScript</h1>
10    <div class="card">
11      <button id="counter" type="button"></
12    </div>
13    <p class="read-the-docs">
14      Click on the Vite and TypeScript logo
15  )})
```

You might say that there is some HTML code in there,

```
TS main.ts U X
6 document.querySelector<HTMLDivElement>('#app')!.innerHTML = `
7   <div>
8     <a href="https://vitejs.dev" target="_blank">
9       | 
10    </a>
11    <a href="https://www.typescriptlang.org/" target="_blank">
12      | 
14    <h1>Vite + TypeScript</h1>
15    <div class="card">
16      <button id="counter" type="button"></button>
17    </div>
18    <p class="read-the-docs">
19      Click on the Vite and TypeScript logos to learn more
20    </p>
21  </div>
```

but in the, that's just some markup

EXPLORER: VITE-DEMO-TS ... JS index-Bd-pKGJy.js X ...

- dist
 - assets
 - JS index-Bd-pKGJy.js
 - # index-Cz4zGhbH.css
 - index.html
 - vite.svg
 - node_modules
 - public
 - src
 - TS counter.ts
 - TS main.ts
 - # style.css
 - typescript.svg

```
1 (function(){const c=document.createElement(
2   <div>
3     <a href="https://vitejs.dev" target="_b
4     
6     <a href="https://www.typescriptlang.org
7     
9     <h1>Vite + TypeScript</h1>
10    <div class="card">
11      <button id="counter" type="button"></
12    </div>
13    <p class="read-the-docs">
14      Click on the Vite and TypeScript logo
15  </p>
```

So in the end, this is JavaScript code here,

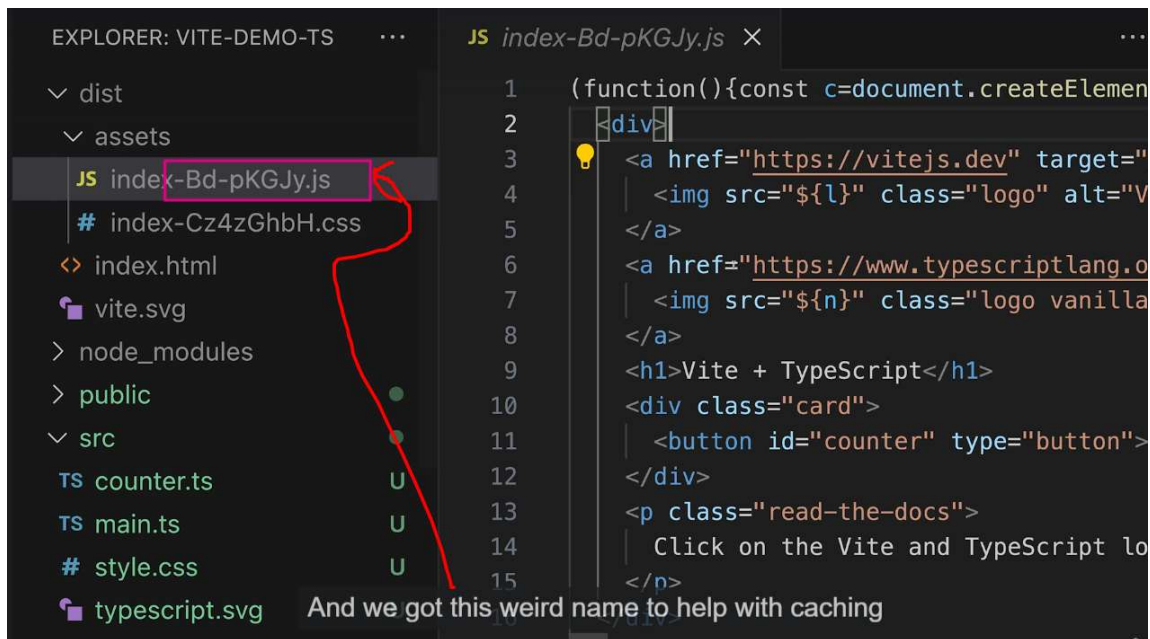
which we got here just already optimized for production.

JS index-Bd-pKGJy.js X ...

```
1 e)if(t.type==="childList")for(const s of t.addedNodes)s.tagName==="LIN
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

So pretty unreadable,

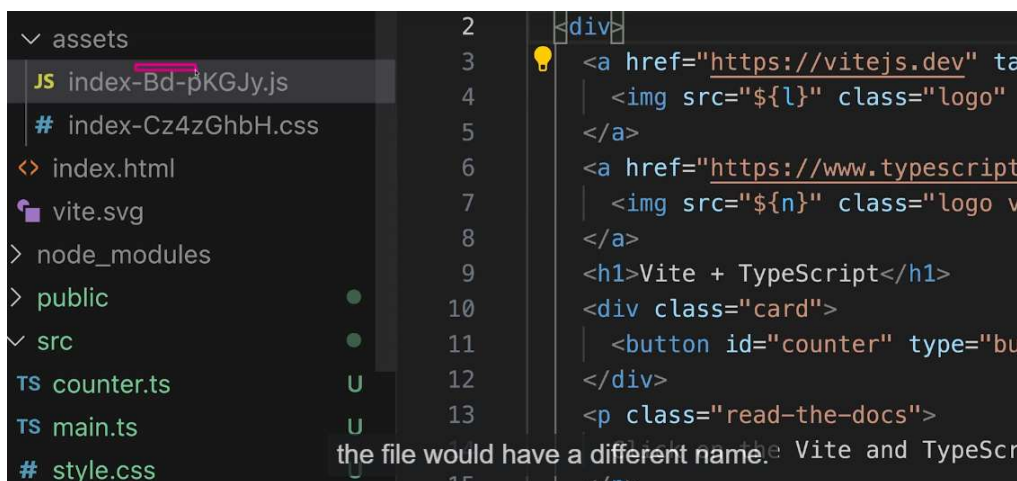
the Vite and TypeScript logos to 1
but therefore also as small as possible.



so that browsers and potentially CDNs

could cache this file.

And if you build and deploy a new version of your project,



And that is what Vite does for us here

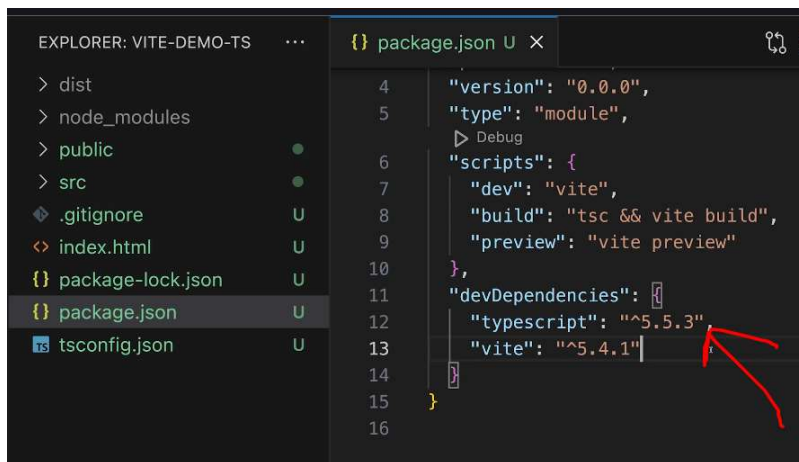
and why you would typically use a tool like Vite

because it helps with building projects

that you can then easily build for production

and which you can then also deploy,

Installing TypeScript on a Per-Project Basis



```
EXPLORER: VITE-DEMO-TS ... {} package.json U X
> dist
> node_modules
> public
> src
> .gitignore
> index.html
{} package-lock.json
{} package.json
ts tsconfig.json

4 "version": "0.0.0",
5 "type": "module",
6 "scripts": {
7   "dev": "vite",
8   "build": "tsc && vite build",
9   "preview": "vite preview"
10 },
11 "devDependencies": {
12   "typescript": "^5.5.3",
13   "vite": "^5.4.1"
14 }
15
16
```

So Vite is useful, but there are a couple of important things about this project that I always like to highlight.

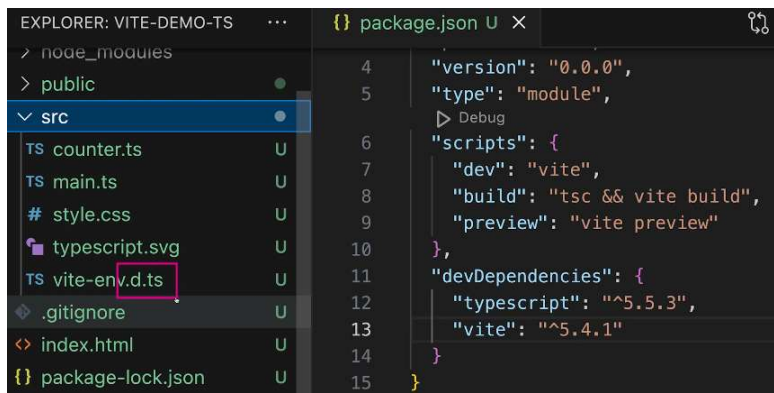
For example, why do we have TypeScript listed as a development dependency in this package.json file? After all, we already installed TypeScript globally earlier in the course. Why install it again here? This is actually a best practice. By adding TypeScript as a project-specific dependency, you gain two major advantages:

No global installation required — You (or anyone else) can build this project without having TypeScript installed globally on your system. The project brings its own version of the TypeScript compiler.

Different TypeScript versions per project — This is especially valuable in real-world scenarios. You might want to use the latest TypeScript version in most projects, but some older projects may only work with a specific older version (e.g., version 4.x). Pinning the version per project allows you to keep using an older version without breaking the project, especially when you don't have time to upgrade it right away.

Making Sense of .d.ts Files

Now, one other thing we had in this project was this .d.ts file.

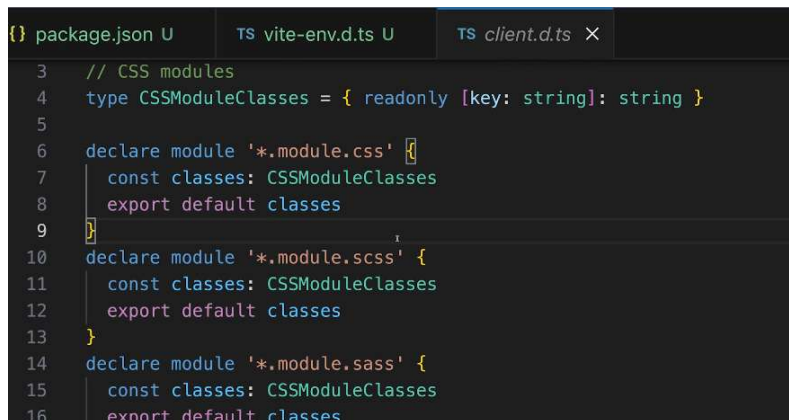


```
EXPLORER: VITE-DEMO-TS ... {} package.json U X
> node_modules
> public
v src
TS counter.ts
TS main.ts
# style.css
typescript.svg
TS vite-env.d.ts
.gitignore
index.html
{} package-lock.json

4 "version": "0.0.0",
5 "type": "module",
6 "scripts": {
7   "dev": "vite",
8   "build": "tsc && vite build",
9   "preview": "vite preview"
10 },
11 "devDependencies": {
12   "typescript": "^5.5.3",
13   "vite": "^5.4.1"
14 }
15
```



```
{ } package.json U TS vite-env.d.ts U X 🔍 ...
1  /// <reference types="vite/client" />
2
```



```
{ } package.json U TS vite-env.d.ts U TS client.d.ts X
3 // CSS modules
4 type CSSModuleClasses = { readonly [key: string]: string }
5
6 declare module '*.module.css' {
7   const classes: CSSModuleClasses
8   export default classes
9 }
10 declare module '*.module.scss' {
11   const classes: CSSModuleClasses
12   export default classes
13 }
14 declare module '*.module.sass' {
15   const classes: CSSModuleClasses
16   export default classes
```

What's that?

Well, that file contains rather strange code,

code that you won't see too often in TypeScript projects

and in the end .d.ts files are TypeScript declaration files

or type declaration files.

And the idea behind such files

is that they don't specify code

that should be executed like main .ts does.

This file here contains a bunch of

JavaScript/TypeScript code

that eventually should be executed after being compiled.

Well, .d.ts files contain just type definitions

that can be used by other files,

but that can't be executed,

so that doesn't compile to JavaScript code,

but that would compile to nothing

because it contains just type definitions, for example.

And this reference thing here,

which is the only code in this .d.ts file,

simply points at some other file

that is part of the vite project.

If you control or command click on vite/client here

you should be taken to this client.d.ts file,

which is part of the vite package,

which is stored in the node modules folder.

And there you see a bunch of type definitions

and this declare thing here,

which in the end just declares TypeScript information.

So that's all just type information,

which can't be executed as JavaScript code.

It's used under the hood to get proper TypeScript support

to avoid errors, to get good IDE support,

but it's not meant to be executed.

You could write .d.ts files on your own

in Project if you knew that you had a file

that only contains type definitions.

But you can also just write a regular .d.ts file

without the .d because you won't get any problems

if you try to compile a file that doesn't lead

to any executable JavaScript code.

We had cases like this before in the course too.

This .d.ts file thing is mostly important for libraries,

so if you're building a library

so that you can ship type definitions

along with your compiled TypeScript code.